

Absolutely. For future revision, you want a **one-page cheat sheet** that reminds you of the entire Frog Jump pattern without reading the full solution again.

Frog Jump — DP Quick Notes

Problem

Given:

```
heights = [10, 20, 30, 10]
```

Frog starts at index `0` and wants to reach index `n-1`.

From index `i`, it can:

- Jump **1 step** → `i-1`
- Jump **2 steps** → `i-2`

Jump cost:

```
abs(heights[i] - heights[j])
```

Goal:

```
Minimum total cost to reach n-1
```

Core Idea

For every index `i`:

```
Answer(i) = minimum cost to reach index i
```

There are only **2 choices**:

```
      i
     / \
    i-1 i-2
```

Therefore:

```
oneJump = answer(i-1) + abs(height[i] - height[i-1])
```

```
twoJump = answer(i-2) + abs(height[i] - height[i-2])
```

This is the **main formula** to remember.

1 Recursion — Top Down

Think:

"To reach `i`, where could I have come from?"

```
FUNCTION recursion(i):  
  
    IF i == 0:  
        RETURN 0  
  
    oneJump =  
        recursion(i-1)  
        + cost(i, i-1)  
  
    twoJump = infinity  
  
    IF i > 1:  
        twoJump =  
            recursion(i-2)  
            + cost(i, i-2)  
  
    RETURN minimum(oneJump, twoJump)
```

Base Case

`i == 0 → 0`

Because you're already standing at index `0`.

Complexity

Time = $O(2^N)$
Space = $O(N)$ ← recursion stack

2 Memoization — Recursion + DP

Problem with recursion

Same subproblem gets calculated multiple times.

```
recursion(3)
  ↓
recursion(2)
  ↓
recursion(1)
```

So store the answer.

Think:

"Have I already solved this index?"

```
FUNCTION memoization(i):

    IF i == 0:
        RETURN 0

    IF dp[i] is already calculated:
        RETURN dp[i]

    oneJump =
        memoization(i-1)
        + cost(i, i-1)

    twoJump = infinity

    IF i > 1:
        twoJump =
            memoization(i-2)
            + cost(i, i-2)

    dp[i] = minimum(oneJump, twoJump)

    RETURN dp[i]
```

Initialization

```
dp = [-1, -1, -1, ..., -1]
```

Complexity

```
Time   = O(N)
Space  = O(N) DP + O(N) recursion stack
```

3 Tabulation — Bottom Up

Now remove recursion.

Think:

"I already know the answer for previous indexes, so calculate from left to right."

```
dp[0] = 0

FOR i = 1 to n-1:

    oneJump =
        dp[i-1]
        + abs(height[i] - height[i-1])

    twoJump = infinity

    IF i > 1:
        twoJump =
            dp[i-2]
            + abs(height[i] - height[i-2])

    dp[i] = minimum(oneJump, twoJump)

RETURN dp[n-1]
```

Complexity

Time = $O(N)$

Space = $O(N)$



The Most Important Revision Pattern

Whenever you see a problem like Frog Jump, ask:

Step 1 — What is my state?

$dp[i]$ = minimum cost to reach i

Step 2 — What are my choices?

$i-1$

$i-2$

Step 3 — What is the cost?

```
abs(heights[i] - heights[j])
```

Step 4 — Recurrence

```
dp[i] = min(  
    dp[i-1] + abs(h[i]-h[i-1]),  
    dp[i-2] + abs(h[i]-h[i-2])  
)
```

Step 5 — Base case

```
dp[0] = 0
```

10-Second Revision

If you forget everything, remember this:

FROG JUMP

State:

$dp[i]$ = minimum cost to reach i

Choices:

1 jump $\rightarrow i-1$

2 jump $\rightarrow i-2$

Cost:

$abs(h[i] - h[j])$

Formula:

```
dp[i] = min(  
    dp[i-1] + abs(h[i]-h[i-1]),  
    dp[i-2] + abs(h[i]-h[i-2])  
)
```

Base:

$dp[0] = 0$

Recursion:

$solve(i-1), solve(i-2)$

Memoization:

$recursion + dp[]$

```
Tabulation:  
  for i = 1 → n-1
```

```
Answer:  
  dp[n-1]
```

DP Conversion Trick

For most basic DP problems, remember:

```
Recursion  
  ↓  
Identify repeated subproblems  
  ↓  
Memoization = Recursion + dp[]  
  ↓  
Reverse the recursion  
  ↓  
Tabulation = Bottom-Up
```

Frog Jump's entire DP logic can essentially be remembered by:

"For `i`, I can come from `i-1` or `i-2`; take the cheaper cost."